Homework 1

Kaushik Veda

1) 1. (25 pts) The speedup is determined by the percentage of the application that can be parallelized and the additional cost of communication between parallel processes. Amdahl's law takes into account the formal but not the latter.

a) What is the speedup with 8 processors if 64% of the application is parallelizable and for every processor used, the communication overhead is 1% of the original execution time?

b) Generalize the speedup formula from part (a) to an arbitrary number of processors, N, and determine the number of processors that will result in the highest speedup in the application.

c) Repeat part (b) assuming that the fraction of the application that is parallelizable is, F, rather than 64%.

a) $T_S = 0.36 + \frac{0.64}{8} + (0.01 \cdot 8) = 0.52$

$S(8) = \frac{1}{0.52} = 1.9230769 \approx 1.92$

b) highest speedup (max fraction value) → smallest denominator

$D(N) = 0.36 + \frac{0.64}{N} + 0.01N \Rightarrow \frac{dD}{dN} = -\frac{0.64}{N^2} + 0.01 \Big|_{N=0}$

$\frac{0.64}{N^2} = 0.01 \Rightarrow N^2 = \frac{0.64}{0.01} \Rightarrow N = 8$ ← # of processors that result in the highest speedup.

Generalized Speedup Formula $= \frac{1}{0.36 + \frac{0.64}{N} + 0.01N}$

c) $D(N) = (1-F) + \frac{F}{N} + 0.01N$

$\frac{dD}{dN} = -\frac{F}{N^2} + 0.01 = 0$

$0.01 = F/N^2 \Rightarrow N^2 = \frac{F}{0.01} \Rightarrow N^2 = 100F$

$N = \sqrt{100F} \Rightarrow N = 10\sqrt{F}$ ← optimal number of processors

2)

2. (25 pts) Recall our on-class discussion of different addressing modes. Among all the nine different addressing modes, which addressing mode(s) is most suitable for the following program variables and operations, and why? (Note that you may list multiple addressing modes for a particular variable/operation type, as long as you can justify the reasons)

a. local variables

b. stack variables

c. array elements

d. pointers

e. branch addresses

f. branch condition evaluation.

a. Local Variables
Displacement (Base) Addressing
Local variables are stored in the stack frame and accessed using a fixed offset from the stack or frame pointer.

b. Stack Variables (Push / Pop)
Auto-Increment / Auto-Decrement Addressing
This mode accesses the stack and updates the stack pointer in one instruction, making push and pop efficient.

c. Array Elements
Scaled or Indexed Addressing
These modes compute addresses as base plus index (and scale), which matches the access pattern of array elements.

d. Pointers
Register Indirect Addressing
Pointers store an address in a register, and this mode directly accesses the memory at that address.

e. Branch Target Addresses
PC-Relative or Register Addressing
PC-relative is used for nearby conditional branches, and register addressing is used when the target address is stored in a register (e.g., function returns).

f. Branch Condition Evaluation
Immediate Addressing
Branch comparisons are usually against constants (especially zero), so immediate mode encodes the constant directly in the instruction.

3)

3. (25 pts). Using the multi-cycle in order pipeline and its latencies we discussed in slide 1 in 4-Control_Hazard.pdf. Further assume there are two register read ports, one write port, and separate I$ and D$ for instruction and data memory accesses. Schedule the following instructions and draw a cycle-wise table similar to slide 15 as we discussed during the class. Mark the data hazard using arrows on the table (also pay attention to structure hazard if any).

fld F4, 0(R2)

fld F5, 8(R2)

fadd.d F6, F4, F5

fadd.d F8, F5, F6

fmult F7, F6, F4

fsd F7 24(R2)

fsd F8 16(R2)

Cycle-wise Schedule

Instr	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
fld F4	IF	ID	EX	MEM	WB													
fld F5		IF	ID	EX	MEM	WB												
fadd F6			IF	ID	S	A1	A2	A3	A4	WB								
fadd F8				IF	S	ID	S	S	S	A1	A2	A3	A4	WB				
fmul F7						IF	S	S	S	ID	M1	M2	M3	M4	M5	M6	M7	WB
fsd F7										IF	ID	S	S	S	S	ID	EX	MEM
fsd F8														IF	ID	EX	MEM	

RAW Hazards

- fld f4 → fadd f6 on Register F4
- fld F5 → fadd f6 on register F5
- fadd f6 → fadd f8 on Register f6
- fadd f6 → fmul f7 on Register f6
- fmul f7 → fsd f7 on Register f7
- fadd f8 → fsd f8 on Register f8

* No Structural Hazards

4)

4. (25 pts) Rename the following code segment (don't forget to also show the overwritten register) assuming the initial map table as shown. The free list starts with p8 and has as many free registers in it as you need to complete the renaming (e.g., p8, p9, p10, …). On your renamed code indicate (by drawing an arrow from the W to the R) how many true data hazards still exist. After renaming, how many instructions have no true data dependencies (and thus could be executed in parallel with enough functional units)? Note that in this example, you are not asked to maintain the free list.

lw $1, 40($6)

$1	p1
$2	p2
$3	p3
$4	p4
$5	p5
$6	p6
$7	P7

lw $2, 60($6)
lw $3, 80($6)

add $4, $2, $1
mul $5, $3, $4
sw $5, 80($7)
sub $3, $4, $5
sw $3, 100($7)
addi $6, $6, 4
addi $7, $7, 4

Data Hazards: (8 data Hazards)

lw p8 → add p11
lw p9 → add p11
lw p10 → mul p12
add p11 → mul p12
mul p12 → sw p12
add p11 → sub p13
mul p12 → sub p13
sub p13 → sw p13

Independent Instructions

lw p8
lw p9
lw p10
addi p14
addi p15

5 independent instructions that could be executed in parallel